

FFT-Based Ciphertext Decomposition and Subring Compression for CKKS

CKKS 密文 $m(X)$ 的同态分解——组内讨论稿

梁朝阳

2026 年 4 月 29 日

摘要

摘要：注意：本文的 section3 两个同态操作具体实现实际已经落后。最新的论文 Hermes 2023 表明有一个工程化更好的拆法。不过 section3 的方法如若想要写在论文中可以作为一种补充。同理 4 的复杂度和噪声分析也过时了。
5 和 6 章节在后续工作中被完成。

目录

1	引言与动机	3
1.1	BGV 与 CoeffToSlot 公式回顾	3
1.2	本人最初想法与 idea 来源	3
1.2.1	idea 解释	3
1.2.2	市面上的拆法	4
1.2.3	我的拆法	5
1.3	当前研究进展	6
2	CKKS 背景	8
2.1	编码与解码	8
2.2	密文	8
2.3	基本操作	8
2.4	迹与积操作	9
2.5	矩阵乘法	9
2.6	传统的 SlotToCoeff	9
3	两个同态操作具体实现（数学推导）	10
3.1	密文的 FFT 分解	10
3.1.1	问题定义与说明	10
3.1.2	记号约定与重要命题	10
3.1.3	正向推导从 ct_P 到两个分支密文	12
3.1.4	反向方向：由两个分支密文恢复原始密文	13
3.2	大环与小环之间的密文转换	15

3.2.1	大环到小环密文压缩	15
3.2.2	压缩后的合并：小环到大环与父层重构	19
4	复杂度与噪声分析	22
4.1	FFT 式奇偶拆分的噪声	22
4.2	大环到小环压缩的噪声	22
4.3	压缩噪声上界	23
4.4	第二行为什么默认不加入输出	23
4.5	压缩是否具有降噪效果	23
4.6	压缩后合并的噪声	24
4.7	层数消耗	24
4.8	复杂度分析	24
4.9	密钥材料复杂度	25
4.10	重复压缩-合并的分布备注	25
4.11	本节总结	25
5	可能的优化方向	25
6	待完成	25
A	平凡矩阵抽取路线	27
A.1	整体置换矩阵	27
A.2	偶次系数抽取矩阵	28
A.3	奇次系数抽取矩阵	28
A.4	为什么本文不采用这条路线	28

1 引言与动机

1.1 BGV 与 CoeffToSlot 公式回顾

虽然我最后的拆分是在 CKKS 上进行的，但是 CKKS 和 BGV/BFV 有很多脱不开的联系。且我最初的想法确实是在 BGV 上进行的，最后因为发现在 CKKS 上可能更好做也有一些参考文献，所以最后转向 CKKS。因此在开始之前先写一下 BGV 公式与 CoeffToSlot 回顾：

一个明文可以写成 $m(X) = \sum_{i=0}^{N-1} m_i X^i \in R_t$ 。密文通常写成二元组 $\mathbf{ct} = (c_0, c_1) \in R_q^2$ 。在私钥 $s(X)$ 下，解密关系可以抽象写成 $c_0(X) + c_1(X)s(X) = m(X) + te(X) \pmod{q}$ 。具体地，BGV/BFV 的解密可以理解为： $\mathbf{ct} \mapsto c_0 + c_1 s = m + te \mapsto m \pmod{t}$ 。

在支持 batching 的参数下，明文环 R_t 可以通过 CRT 分解为若干槽： $R_t \simeq \prod_{i=0}^{N-1} \mathbb{F}_t$ 。于是，一个明文多项式 $m(X) \in R_t$ 既可以用系数形式表示： $m(X) = m_0 + m_1 X + \dots + m_{N-1} X^{N-1}$ ，也可以用槽形式表示： $\mathbf{m} = (m(\alpha_0), m(\alpha_1), \dots, m(\alpha_{N-1}))$ ，其中 α_i 是对应的明文环分解点。

因此存在一个线性变换 $M : R_t \rightarrow \mathbb{F}_t^N$ ，把系数表示转换成槽表示：

$$M : m(X) \mapsto (m(\alpha_0), m(\alpha_1), \dots, m(\alpha_{N-1}))$$

它的逆变换为 $M^{-1} : \mathbb{F}_t^N \rightarrow R_t$ ，把槽表示重新插值回系数表示。

1.2 本人最初想法与 idea 来源

我最初是在读完 FHE 的 CKKS 和 BGV/BFV 基础论文后就开始赶热点。之后就接触到了 Bootstrapping 具体内部可以优化的东西（比如 CoeffToSlot 的优化是我感觉比较火的一个热点。比如 [1]）然后就开始阅读 BGV 的 Bootstrapping 中的 CoeffToSlot[2]。其中很多人都在优化的一个操作是： $M^{-1}UM$ 这三个东西对 $m(X)$ 的同态乘法，形式为：

$$m(X) \mapsto Mm(X) \mapsto UMm(X) \mapsto M^{-1}UMm(X).$$

其中对 CoeffToSlot 中的一个矩阵 U 的操作研究的已经较为成熟了 [3]，也就是继续沿着优化某个操作的路子基本已经很深了。再往下做可能难度会非常大，即使做出来也有概率没有效果。而对 M 本身也有研究。因此我就把想法放到了 $m(X)$ 上。最初的核心想法就是用存储换并行和复杂度降低之类的。

1.2.1 idea 解释

对 $m(X)$ 做分解听起来很自然，但是实际遇到的问题非常多。首先就是“怎么拆”的问题。我目前阅读到的拆明文的有两篇，其中 2026 年的一篇预印本明确表达了自己在拆明文。另一篇 PaCo 做的也是拆明文的事情但是他没有明确写出来自己的切入角度是拆明文。

如果只是最朴素地在明文阶段把 $m(X)$ 拆成若干个小多项式，然后分别加密，那么这个想法基本没有太大意义，甚至可能带来负效果。主要原因如下。

会破坏 SIMD 结构。 在 BGV/BFV 的 batching 设置中， $m(X) \in R_t$ 不是普通系数数组，而是多个 slot 编码后的多项式。若直接按系数拆分，可能破坏原有 SIMD 布局，使后续 slot-wise 操作变得更复杂。

拆开后仍然需要合并。即使把 $m(X)$ 拆成 $m_1(X), m_2(X), \dots$, 也不代表可以分别计算后简单合并。对于乘法或 bootstrapping 中的非线性步骤, 拆分会产生交叉项, 合并成本可能很高。

密文数量增加, 但环维度没有下降。 原来只需要一个密文 $\text{ct} = \text{Enc}(m)$ 。若拆成 r 份, 就需要 r 个密文。更关键的是, 如果这些密文仍然都在同一个大环 R_q 中, 那么只是把一个大环密文变成多个大环密文, 并没有真正降低单个密文的计算维度。

任意拆分未必能被后续算法利用。 CoeffToSlot、SlotToCoeff、卷积和 bootstrapping 都有各自的代数结构。如果拆分方式与这些结构不相容, 反而可能引入额外的 repacking 或线性变换成本。

因此, 真正有意义的拆分不应该是在明文阶段随意拆开再分别加密。关键问题不只是“能不能拆”, 而是拆分后能否保持可利用的代数结构, 并且真正降低密文所在的环维度。否则, 拆分只会增加密文数量和计算复杂度, 却没有带来实际收益。

1.2.2 市面上的拆法

按进制分解 设明文 $m \in \mathbb{Z}_p$, 其中 p 很大, 例如 2^{64} 、大素数、或者一般的大整数模数。此时最自然的分解方式是**基数/分枝分解 (radix/limb decomposition)**。

设底数为 b , 位数为 d , 大致满足 $p \approx b^d$ 。对 $m \in \mathbb{Z}_p$, 最朴素的分解为

$$m = \sum_{i=0}^{d-1} b^i m_i, \quad m_i \in \{0, \dots, b-1\}.$$

这是一种最简单的分解。但是好像没啥用。这个分解在 BFV 的重现性化中用到了, 也就是把 r1k 拆成 2 进制然后来降低噪声, 我记忆中自己尝试了一下, 但是记忆中好像尝试结果不尽人意。

2026 年: dBFV 的放松数码分解 (Relaxed Digit Decomposition) [4]

dBFV 的关键不在于唯一的 digit 表示, 而在于将明文嵌入一个**分解环**中。定义基向量 $\mathbf{b} = (1, b, b^2, \dots, b^{d-1}) \in \mathbb{Z}_p^d$. 定义分解格 (论文中的 lattice) $L_b := \{z(B) \in \mathbb{Z}[B] : z(b) \equiv 0 \pmod{p}\}$. 由此得到环同构

$$\mathbb{Z}_p \cong \mathbb{Z}[B]/L_b,$$

同构由 $[z(B)] \mapsto z(b) \pmod{p}$ 给出。dBFV 论文正是这样定义的: 将 $\mu \in \mathbb{Z}_p$ 视为某个多项式余类 $\mu(B) \in \mu + L_b$, 并强调 evaluation at b 给出了环同构。这个环同构意味着, 我们可以用 $\mathbb{Z}[B]$ 中的多项式 (系数不必严格小于 b) 来代表 $\mathbb{Z}_p$ 中的元素, 而模去 L_b 则确保了求值在 $B = b$ 时得到正确结果。这是将值域问题转化为多项式代数问题的关键一步。

他这个我没细看, 他自己声称实现了很高的吞吐量之类的。但是我看就加密一个在 $\mathbb{Z}_p$ 上的 m , 感觉是因为他不会拆多项式的 $m(X)$ 所以才转而拆 m 的。

我也没特别注意看他是在密文拆的还是直接拆的明文, 但大概率也是在同态的情况下对密文操作, 而不是先拆 m , 然后分别发送 m_i 的同态密文。

1.2.3 我的拆法

既然已经有人做了整数的情况，则我考虑的是多项式的情况，多项式要比整数复杂很多，但是可操作的代数结构也更丰富。设明文为多项式 $m(X) \in R_N := \mathbb{Z}_t[X]/(X^N + 1)$ ，其中 $N = 2^k$ 是 2 的幂次。此时最自然的分解方式有两种：

- 系数分块分解 (coefficient-block decomposition)；
- 基于 FFT 的奇偶分解 (FFT even-odd decomposition)。

第二种与 CoeffToSlot / SlotToCoeff 的 FFT-like 分解最为接近。Geelen 等人的工作正是在 2 的幂次分圆环上将 slot-to-coefficient transformation 分解为多阶段 FFT-like 线性变换，从而将复杂度从线性级别降至对数级阶段。

系数分块分解 设 $R_N = \mathbb{Z}_t[X]/(X^N + 1)$ ，并设 $N = sb$ 。则任意 $m(X) \in R_N$ 均可唯一地写成

$$m(X) = \sum_{j=0}^{s-1} X^{jb} m_j(X), \quad \deg m_j < b.$$

这就是将 $m(X)$ 按长度为 b 的系数块分解为 s 段。可以将它视为向量 $\Phi_{\text{blk}}(m) = (m_0, \dots, m_{s-1})$ 。这是一个线性分解，本质上是**模的直和分解**，而非 dBFV 那种商环上的数码分解。

- **加法推导**：考虑 $m(X) = \sum_{j=0}^{s-1} X^{jb} m_j(X)$, $n(X) = \sum_{j=0}^{s-1} X^{jb} n_j(X)$ 。则 $m(X) + n(X) = \sum_{j=0}^{s-1} X^{jb} (m_j(X) + n_j(X))$ 因此在分块分解下，加法是**逐块独立的**： $\Phi_{\text{blk}}(m + n) = (m_0 + n_0, \dots, m_{s-1} + n_{s-1})$ 。这一点非常干净。
- **乘法推导**：块卷积首先在普通多项式环中展开： $m(X)n(X) = \sum_{i=0}^{s-1} \sum_{j=0}^{s-1} X^{(i+j)b} m_i(X)n_j(X)$ 。令 $h_u(X) = \sum_{i+j=u} m_i(X)n_j(X)$, $0 \leq u \leq 2s-2$ 。则 $m(X)n(X) = \sum_{u=0}^{2s-2} X^{ub} h_u(X)$ 。这说明：**系数分块分解下的乘法，本质是块卷积**，而非逐块乘法。也就是说，块 u 依赖于所有满足 $i + j = u$ 的块对。

因为交叉项太多。如果 s 个块都互相参与，则总耦合结构就是一个块 Toeplitz 卷积。

所以系数分块分解更适合：blockwise Hadamard；局部卷积；局部特征计算；partial transform。而不适合通用乘法或者强耦合的运算。

比如如果定义的任务不是环乘法，而是 Hadamard 或局部内积。最后再解密后聚合。”在这种任务上，系数分块分解非常自然： $f(m, n) = \sum_{j=0}^{s-1} X^{jb} (m_j(X) \odot n_j(X))$ 或者更简单地，按块做某个局部算子： $f(m, n) = (\phi(m_0, n_0), \dots, \phi(m_{s-1}, n_{s-1}))$ ，那么此时每个块之间没有交叉项。这正是前文所述：“某些函数可能只需要局部 Hadamard 或局部内积，最后再解密后聚合。”在这种任务上，系数分块分解非常自然。

基于 FFT 的奇偶分解 设 $R_{2n} = \mathbb{Z}_t[X]/(X^{2n} + 1)$ 。任意多项式 $m(X) \in R_{2n}$ 均可唯一分解为

$$m(X) = m_0(X^2) + X m_1(X^2),$$

其中 $\deg m_0, \deg m_1 < n$ 。写成显式系数形式：若

$$m(X) = \sum_{k=0}^{2n-1} a_k X^k,$$

则

$$m_0(Y) = \sum_{r=0}^{n-1} a_{2r} Y^r, \quad m_1(Y) = \sum_{r=0}^{n-1} a_{2r+1} Y^r,$$

其中 $Y = X^2$ 。这种分解可以递归，也更加自然。

- **加法推导：** 设 $m(X) = a(Y) + Xb(Y)$, $n(X) = c(Y) + Xd(Y)$, 其中 $Y = X^2$ 。则 $m(X) + n(X) = (a(Y) + c(Y)) + X(b(Y) + d(Y))$ 。因此奇偶分解下，加法就是 $(a, b) + (c, d) = (a + c, b + d)$ ，仍然是分量级线性。
- **乘法推导：** 设 $m(X) = a(Y) + Xb(Y)$, $n(X) = c(Y) + Xd(Y)$, 其中 $Y = X^2$ 。则 $m(X)n(X) = (a(Y) + Xb(Y))(c(Y) + Xd(Y)) = ac(Y) + X(ad(Y) + bc(Y)) + X^2bd(Y)$ 。因此奇偶分解下，乘法就是 $(a, b) \star (c, d) = (ac(Y) + Ybd(Y), ad(Y) + bc(Y))$ 。

补充说明： 该公式中的 Y 项来自于 $X^2 = Y$ 的替换，这反映了在商环 R_{2n} 中乘法导致的“扭曲”效应。这正是快速傅里叶变换中旋转因子（twiddle factor）的来源之一。

这个乘法规则说明：

- 奇偶分解将一个大环 R_{2n} 的运算重写为较小环 R_n 上的**两分量递归运算**。
- 这里的 $Y = X^2$ 项是 butterfly 里“旋转因子”的来源之一。

因此我感觉这个分解可能很贴合 coefficient-to-slot; DFT/FFT-like linear transforms; sparse packing / subring transforms。

1.3 当前研究进展

实际研究中，我目前研究了对 CKKS 密文的分解，因为一般来讲 FFT 分解对 CKKS 是更加契合的。研究完后发现这套思路可能在 BGV/BFV 上也能做。不过鉴于 PaCo 那篇也是在 CKKS 上做分解，所以我在 CKKS 上做还算有参考，如果在 BGV 上做就一点参考都没有了。

再加上最最重要的是，CKKS 的 CoeffToSlot 有把一个密文拆成两个密文的先例，而且是非常经典的操作。

目前已经解决了两个相互衔接的基本问题：

1. 如何在密文状态下把 $m(X)$ 按 FFT 拆开
2. 拆开以后如何真正把密文搬到更小的环里

其中，第二个问题实际推导下来比第一个问题难非常多，第一个问题也不简单，但是有看似平凡的做法，之所以是看似原因是实际上不行。但是这对我实际推导有很大帮助。

密文下对 $m(X)$ 按 FFT 分解的形式化定义 CKKS 密文上执行的“FFT 式奇偶拆分 / 反向重构”过程。设

$$P(Y) = \sum_{j=0}^{2n-1} p_j Y^j \in \mathbb{R}[Y]/(Y^{2n} + 1),$$

我们希望在~~不经过 CoeffToSlot / SlotToCoeff 的前提下~~，直接从一个加密 $P(Y)$ 的 CKKS 密文出发，得到两个分别对应

$$P_{\text{even}}(T) = \sum_{i=0}^{n-1} p_{2i} T^i, \quad P_{\text{odd}}(T) = \sum_{i=0}^{n-1} p_{2i+1} T^i$$

的密文，并进一步研究如何把这两个分支密文真正压缩到更小的环中。

如何实现“大环到小环密文压缩” 这个概念可能没那么好理解，我想解决的是：将一个大环 $R_N = \mathbb{Z}_q[X]/(X^N + 1)$ 上的 CKKS 密文（其明文为 $m(X^2)$ ，其中 $m \in R_n = \mathbb{Z}_q[Y]/(Y^n + 1)$ 且 $N = 2n$ ）同态地转换为一个小环 R_n 上的密文（其明文为 $m(Y)$ ）。整个过程仅使用公开的线性映射 τ （取偶次项系数）和模块到环的密钥切换技术。

例如：为了直观理解这个压缩过程，可以先看一个最简单的例子。设

$$R_N = \mathbb{Z}_q[X]/(X^N + 1), \quad R_n = \mathbb{Z}_q[Y]/(Y^n + 1), \quad N = 2n,$$

并令 $Y = X^2$ 。假设大环密文为 $\mathbf{ct} = (c_0(X), c_1(X)) \in R_N^2$ ，并且它解密后表示 $m(X^2)$ 。也就是说，

$$c_0(X) + s(X)c_1(X) = \Delta m(X^2) + e(X).$$

这里的关键是：虽然明文只含有 X 的偶次幂，

$$m(X^2) = m_0 + m_1 X^2 + m_2 X^4 + \cdots + m_{n-1} X^{2n-2},$$

但是密文分量 $c_0(X), c_1(X)$ 仍然是大环中的多项式：

$$c_0(X) = a_0 + a_1 X + a_2 X^2 + \cdots + a_{2n-1} X^{2n-1},$$

$$c_1(X) = b_0 + b_1 X + b_2 X^2 + \cdots + b_{2n-1} X^{2n-1}.$$

因此，压缩之前的密文是两个长度约为 $2n$ 的多项式： $\mathbf{ct} = (c_0, c_1) \in R_N^2$ 。压缩的目标是构造一个小环密文 $\mathbf{ct}' = (d_0(Y), d_1(Y)) \in R_n^2$ ，使得

$$d_0(Y) + s'(Y)d_1(Y) = \Delta m(Y) + e'(Y).$$

也就是说，压缩之后密文仍然是二元组，但每个分量从大环多项式变成了小环多项式：

$$(c_0(X), c_1(X)) \in R_N^2 \quad \mapsto \quad (d_0(Y), d_1(Y)) \in R_n^2.$$

从数组长度上看，就是

$$\text{两个长度 } 2n \text{ 的多项式} \quad \mapsto \quad \text{两个长度 } n \text{ 的多项式}.$$

所以所谓“大环到小环密文压缩”，并不是把密文分量 c_0, c_1 简单截断，而是重新构造一个新的小环密文 $\mathbf{ct}'$ 。它解密后仍然表示同一个消息，只是消息从大环中的 $m(X^2)$ 变成了小环中的 $m(Y)$ 。

2 CKKS 背景

由于本文的分解等都是在 CKKS 上进行，所以需要对 CKKS 的理解要求高一些。这里给出 PaCo 那篇文章对 CKKS 的总结，我自己读完很受益：

我们现在回顾 CKKS 方案的关键组成部分。

2.1 编码与解码

CKKS 支持对复向量 $\mathbf{z} \in \mathbb{C}^n$ 的加密，其中 n 是 N 的真因子。这是通过将向量 $\mathbf{z} \in \mathbb{C}^n$ 解释为多项式 $m \in \mathbb{Z}[Y]/(Y^{2n} + 1)$ 来实现的。为此，我们考虑环同构：

$$\tau_n : \frac{\mathbb{R}[Y]}{(Y^{2n} + 1)} \ni p \mapsto (p(\zeta_{n,i}))_{i \in [0, n)} \in \mathbb{C}^n,$$

其中 $\zeta_{n,i} = \exp(2\pi i \cdot 5^i / (4n))$ 是 $(4n)$ -次本原根的一半。

对于固定的正整数 Δ ，称为 CKKS 方案的缩放因子，我们通过下式编码复向量 $\mathbf{z} \in \mathbb{C}^n$ ：

$$\text{Ecd}(\mathbf{z}) = \lfloor \Delta \tau_n^{-1}(\mathbf{z}) \rfloor \in \frac{\mathbb{Z}[Y]}{(Y^{2n} + 1)};$$

通过令 $Y = X^{N/(2n)}$ ，我们将 $\text{Ecd}(\mathbf{z})$ 视为 $\mathbb{Z}[X]/(X^N + 1)$ 中的元素。因此，对于两个真因子 $n \geq B$ 的 N 以及两个向量 $\mathbf{z}_1 \in \mathbb{C}^B$ 和 $\mathbf{z}_0 = (\mathbf{z}_1)_{i \in [0, n/B]} \in \mathbb{C}^n$ ，有 $\text{Ecd}(\mathbf{z}_0) = \text{Ecd}(\mathbf{z}_1)$ 。因此，通过自然嵌入 $\mathbb{C}^B \ni \mathbf{z}_1 \mapsto (\mathbf{z}_1)_{i \in [0, n/B]} \in \mathbb{C}^n$ ，将 $\mathbb{C}^B$ （装备 Hadamard 积）视为 $\mathbb{C}^n$ 的子环是合理的。

解码多项式 $m \in \mathbb{Z}[Y]/(Y^{2n} + 1)$ 的公式为：

$$\text{Dcd}_n(m) = \frac{\tau_n(m)}{\Delta} \in \mathbb{C}^n.$$

对于两个复向量 $\mathbf{z}_0, \mathbf{z}_1 \in \mathbb{C}^n$ ，有 $\text{Ecd}(\mathbf{z}_0) + \text{Ecd}(\mathbf{z}_1) \approx \text{Ecd}(\mathbf{z}_0 + \mathbf{z}_1)$ 。定义运算 $m_0 \times m_1 = [m_0 m_1 / \Delta] \in \mathbb{Z}[X]/(X^N + 1)$ ，其中 $m_0, m_1 \in \mathbb{Z}[X]/(X^N + 1)$ ，进一步有 $\text{Ecd}(\mathbf{z}_0) \times \text{Ecd}(\mathbf{z}_1) \approx \text{Ecd}(\mathbf{z}_0 \odot \mathbf{z}_1)$ 。

2.2 密文

一个 CKKS 密文在模数 q 下加密多项式 $m \in \mathbb{Z}[X]/(X^N + 1)$ ，是一个元组 $\mathbf{ct} = (\mathbf{ct}_0, \mathbf{ct}_1)$ ，其中 $\mathbf{ct}_i \in \mathbb{Z}[X]/(X^N + 1)$ 满足解密方程 $\mathbf{ct}_0 + s \cdot \mathbf{ct}_1 = m + e \bmod q$ ，这里 $e \in \mathbb{Z}[X]/(X^N + 1)$ 是小误差， $s \in \mathbb{Z}[X]/(X^N + 1)$ 是秘密密钥。记此加密为 $\mathbf{ct} = \text{Euc}(m)$ ，若 $m \approx \text{Ecd}(\mathbf{z})$ ，则记作 $\mathbf{ct} = \text{Euc}^*(\mathbf{z})$ 。

2.3 基本操作

表 1 列出了 CKKS 支持的主要同态操作。

表 1: CKKS 中的基本同态操作开销

操作	符号	加密向量	消耗级别	操作	符号	加密向量	消耗级别
重缩放	$\text{RS}(\text{ct})$	$\mathbf{z}/\Delta$	1	加法	$\text{ct}_0 + \text{ct}_1$	$\mathbf{z}_0 + \mathbf{z}_1$	0
减法	$\text{ct}_0 - \text{ct}_1$	$\mathbf{z}_0 - \mathbf{z}_1$	0	明文-密文乘法	$\text{Ecd}(\mathbf{w}) \times \text{ct}$	$\mathbf{w} \odot \mathbf{z}$	1
标量-密文乘法	$\text{Ecd}(\lambda) \times \text{ct}$	$\lambda \cdot \mathbf{z}$	1	密文-密文乘法	$\text{ct}_0 \times \text{ct}_1$	$\mathbf{z}_0 \odot \mathbf{z}_1$	1
按 j 个槽位旋转	$\text{Rot}_j(\text{ct})$	$\text{Rot}_j(\mathbf{z})$	0	共轭	$\text{Conj}(\text{ct})$	$\bar{\mathbf{z}}$	0

2.4 迹与积操作

定义向量 $\mathbf{z} = (z_i)_{i \in [0, n]} \in \mathbb{C}^n$ 的迹与积操作:

$$\begin{aligned} \text{Tr}_{n \rightarrow B}(\mathbf{z}) &= \left(\sum_{j=0}^{n/B-1} z_{jB+i} \right)_{i \in [0, B)} \in \mathbb{C}^B, \\ \text{Pr}_{n \rightarrow B}(\mathbf{z}) &= \left(\prod_{j=0}^{n/B-1} z_{jB+i} \right)_{i \in [0, B)} \in \mathbb{C}^B. \end{aligned}$$

2.5 矩阵乘法

对于矩阵 $\mathbf{A} = (A_{i,j})_{i,j \in [0, n]} \in \mathbb{C}^{n \times n}$, 定义第 j 条对角线: $\text{Diag}_j(\mathbf{A}) = (A_{i, [i+j]_n})_{i \in [0, n]}$. 矩阵-向量乘积可表示为:

$$\mathbf{A} \cdot \mathbf{z} = \sum_{j=0}^{n-1} \text{Diag}_j(\mathbf{A}) \odot \text{Rot}_j(\mathbf{z}).$$

表 2 列出了更多同态操作。

表 2: CKKS 中的更多同态操作

操作	符号	加密向量	消耗级别
迹	$\text{Tr}_{n \rightarrow B}(\text{ct})$	$\text{Tr}_{n \rightarrow B}(\mathbf{z})$	0
积	$\text{Pr}_{n \rightarrow B}(\text{ct})$	$\text{Pr}_{n \rightarrow B}(\mathbf{z})$	$\log(n/B)$
矩阵-密文乘法	$\text{Ecd}(\mathbf{A}) \times \text{ct}$	$\mathbf{A} \cdot \mathbf{z}$	1

2.6 传统的 SlotToCoeff

考虑复向量 $\mathbf{z} \in \mathbb{C}^n$ 与对应多项式 $p = \sum_{i=0}^{2n-1} p_i Y^i = \tau_n^{-1}(\mathbf{z}) \in \mathbb{R}[Y]/(Y^{2n} + 1)$. 定义 $\mathbf{p}_0 = (p_i)_{i \in [0, n]}$ 和 $\mathbf{p}_1 = (p_{i+n})_{i \in [0, n]}$, 范德蒙矩阵 $U_n = (\zeta_{n,i}^j)_{i,j \in [0, n]}$ 有:

$$\mathbf{z} = U_n(\mathbf{p}_0 + \mathbf{I} \cdot \mathbf{p}_1). \quad (3)$$

给定密文 $\text{ct}_i = \text{Enc}^*(\mathbf{p}_i)$, $i \in [0, 2)$, 我们可同态地得到:

$$\text{SlotToCoeff}_n(\text{ct}) = \text{Enc}^*(\mathbf{z}).$$

此操作被称为 SlotToCoeff, 因为它将槽位包含多项式 p 的系数的向量的加密, 转换为系数由 p 的系数 (乘以 Δ) 给出的多项式的加密。

3 两个同态操作具体实现（数学推导）

3.1 密文的 FFT 分解

3.1.1 问题定义与说明

这一个章节的数学推导目标是严格说明以下两件事：**1. 正向拆分**：从一个加密的 $P(Y) = \sum_{j=0}^{2n-1} p_j Y^j$ CKKS 密文出发，同态地得到两个分支密文，分别对应 FFT 递归中的偶部分和奇部分；**2. 反向重构**：从这两个分支密文出发，同态地恢复原始的 $P(Y)$ 密文。

形式化地：考虑：

$$P(Y) = \sum_{j=0}^{2n-1} p_j Y^j \in \mathbb{R}[Y]/(Y^{2n} + 1),$$

我们希望在不经 CoeffToSlot / SlotToCoeff 的前提下，直接从一个加密 $P(Y)$ 的 CKKS 密文出发，得到两个分别对应 $P_{\text{even}}(T) = \sum_{i=0}^{n-1} p_{2i} T^i$, $P_{\text{odd}}(T) = \sum_{i=0}^{n-1} p_{2i+1} T^i$ 的密文；并进一步给出从这两个分支密文重构原始密文的完整公式。

需要强调的是，这里我们**不采用**直接把系数向量

$$(p_0, p_1, \dots, p_{2n-1})^\top$$

看成一个长度 $2n$ 向量，再用矩阵 E, O 去做抽取的方案。原因是：标准 CKKS 的“矩阵-密文乘法”作用对象本质上是**槽向量表示**，而这里我们要处理的是**多项式系数表示**。如果走矩阵 E, O 的路线（**这个路线具体是什么意思我会放到附录中**），通常需要先做一次 CoeffToSlot，把系数信息变成槽向量。

这样做虽然理论上也可行，但开销巨大，因为 CoeffToSlot 是 Bootstrapping 里面开销非常大的操作。所以我的操作实际上绕开了 CoeffToSlot，只在多项式上实现拆分。总结下关键难点，这两个问题都是非平凡的。：**1. 如何在密态的情况下提取系数并做分离？**；**2. 如何把一个密文变成两个？**

3.1.2 记号约定与重要命题

记号约定。固定大环维度 N ，并假设 N 为 2 的幂。取 $2n \mid N$ ，令 $k = \frac{N}{2n}$ 。本文在 $\mathcal{S}_n = \mathbb{R}[Y]/(Y^{2n} + 1)$, $\mathcal{R}_N^{\mathbb{R}} = \mathbb{R}[X]/(X^N + 1)$ 中工作。通过代换 $Y = X^k$ ，可以把 $\mathcal{S}_n$ 嵌入到 $\mathcal{R}_N^{\mathbb{R}}$ 中；这是良定义的，因为 $(X^k)^{2n} = X^N = -1$ 。

设 $P(Y) = \sum_{j=0}^{2n-1} p_j Y^j \in \mathcal{S}_n$ 。定义 $P_{\text{even}}(T) = \sum_{i=0}^{n-1} p_{2i} T^i$, $P_{\text{odd}}(T) = \sum_{i=0}^{n-1} p_{2i+1} T^i$ 。由于 $T = Y^2$ 满足 $T^n = Y^{2n} = -1$ ，所以 $P_{\text{even}}(T), P_{\text{odd}}(T) \in \mathbb{R}[T]/(T^n + 1)$ 。于是 $P(Y) = P_{\text{even}}(Y^2) + Y P_{\text{odd}}(Y^2)$ 。

CKKS 语义记号约定 为了把推导写得清楚，同时不被实现细节淹没，约定如下**语义记号**。

定义 3.1 (密文表示的消息). 设当前 CKKS *scale* 为 Δ 。若一个密文 $\mathbf{ct}$ 解密后得到的明文多项式为

$$m(X) = \text{Round}(\Delta F(X)) + e(X) \in \mathcal{R}_N^{\mathbb{Z}},$$

其中 $F(X) \in \mathcal{R}_N^{\mathbb{R}}$ ，而 $e(X)$ 是 CKKS 中通常的小噪声项，则记为

$$\mathbf{ct} \models_{\Delta} F(X).$$

这表示： $\mathbf{ct}$ 在当前 *scale* 下表示消息 $F(X)$ 。

- 当写 $\mathbf{ct} \models_{\Delta} F(X)$ 时，真正被加密的是 $\text{Round}(\Delta F(X))$ （再加上通常的小加密噪声）；
- 本文中的代数等式都作用在 $F(X)$ 这个**被表示的消息**层面；
- 若实现中需要做 *rescale*、*modulus switching* 或 *scale matching*，则默认按标准 CKKS 方式处理，使得 $\mathbf{ct}$ 所表示的消息不变。为集中讨论代数正确性，这些实现细节在公式中省略。

所用到的四种同态操作 推导只使用以下四类标准操作。

- (i) **自同构**： $\text{Auto}_a(\mathbf{ct})$ ，其中 a 为奇数，表示对密文应用环自同构 $\sigma_a : \mathcal{R}_N^{\mathbb{R}} \rightarrow \mathcal{R}_N^{\mathbb{R}}, f(X) \mapsto f(X^a)$ 。本文记号 Auto_a 默认包含标准 Galois key switching，即自同构后密文被切换回原私钥下。
- (ii) **加法**： $\text{Add}(\mathbf{ct}_1, \mathbf{ct}_2)$ 。
- (iii) **减法**： $\text{Sub}(\mathbf{ct}_1, \mathbf{ct}_2)$ 。
- (iv) **明文-密文乘法**： $\text{MulPt}_{u(X)}(\mathbf{ct})$ ，其中 $u(X) \in \mathcal{R}_N^{\mathbb{R}}$ 是一个明文多项式（例如常数 $1/2$ 、单项式 X^k 或 $-X^{N-k}$ ）。

这些操作在消息层面满足以下标准语义：

命题 3.2 (操作的消息语义). 设 $\mathbf{ct}, \mathbf{ct}_1, \mathbf{ct}_2$ 为 CKKS 密文， $u(X) \in \mathcal{R}_N^{\mathbb{R}}$ ， a 为奇数。

(1) 若 $\mathbf{ct} \models_{\Delta} F(X)$ ，则

$$\text{Auto}_a(\mathbf{ct}) \models_{\Delta} F(X^a).$$

(2) 若 $\mathbf{ct}_1 \models_{\Delta} F_1(X)$ 且 $\mathbf{ct}_2 \models_{\Delta} F_2(X)$ ，则

$$\text{Add}(\mathbf{ct}_1, \mathbf{ct}_2) \models_{\Delta} F_1(X) + F_2(X),$$

$$\text{Sub}(\mathbf{ct}_1, \mathbf{ct}_2) \models_{\Delta} F_1(X) - F_2(X).$$

(3) 若 $\mathbf{ct} \models_{\Delta} F(X)$ ，则

$$\text{MulPt}_{u(X)}(\mathbf{ct}) \models_{\Delta} u(X) F(X).$$

注 3.3. 上面的第三条在实现里可能伴随 *scale* 调整与 *rescale*；但在“被表示的消息”这一层面，其作用就是乘上 $u(X)$ 。

上标 $\uparrow$ 的含义: 上标 $\uparrow$ 表示“提升回大环中的表示”。稍后我们会得到形如:

$$P_{\text{even}}(X^{2k}), \quad P_{\text{odd}}(X^{2k})$$

这样的对象。它们虽然仍属于大环 $\mathcal{R}_N^{\mathbb{R}}$, 但只包含 X^{2k} 的幂, 因此实际上落在更小的子环里。为提醒读者“它仍存放在大环中, 但语义上来自更小子环”, 我们记对应密文为

$$\text{ct}_{\text{even}}^{\uparrow}, \quad \text{ct}_{\text{odd}, Y}^{\uparrow}, \quad \text{ct}_{\text{odd}}^{\uparrow}.$$

多项式与偶奇分支

设 $P(Y) = \sum_{j=0}^{2n-1} p_j Y^j \in \mathcal{S}_n$ 。定义 $P_{\text{even}}(T) = \sum_{i=0}^{n-1} p_{2i} T^i$, $P_{\text{odd}}(T) = \sum_{i=0}^{n-1} p_{2i+1} T^i$ 。于是 $P(Y) = P_{\text{even}}(Y^2) + Y P_{\text{odd}}(Y^2)$ 。

命题 3.4 (FFT 式偶奇分解). 设 $P(Y) = \sum_{j=0}^{2n-1} p_j Y^j$, 定义 $P_{\text{even}}, P_{\text{odd}}$ 如上, 则 $P(Y) = P_{\text{even}}(Y^2) + Y P_{\text{odd}}(Y^2)$ 。

命题 3.5 (以 $P(-Y)$ 表示偶奇分支). 在上述等式下, 有 $P(-Y) = P_{\text{even}}(Y^2) - Y P_{\text{odd}}(Y^2)$ 。从而:

$$\begin{aligned} P_{\text{even}}(Y^2) &= \frac{P(Y) + P(-Y)}{2}, \\ Y P_{\text{odd}}(Y^2) &= \frac{P(Y) - P(-Y)}{2} \end{aligned}$$

3.1.3 正向推导从 ct_P 到两个分支密文

输入 设输入密文为

$$\text{ct}_P \models_{\Delta} P(X^k), \quad P(X^k) = \sum_{j=0}^{2n-1} p_j X^{jk} \in \mathcal{R}_N^{\mathbb{R}}.$$

也就是说, ct_P 是一个直接在 CKKS 大环中表示 $P(Y)$ (经 $Y = X^k$ 嵌入后) 的密文。

第一步: 构造 $P(-Y)$ 的密文

命题 3.6 (实现 $Y \mapsto -Y$ 的自同构). 取 $a = 1 + 2n$ 。由于本文中 N 为 2 的幂, $2N$ 也是 2 的幂, 因此任意奇数都与 $2N$ 互素。于是 a 在模 $2N$ 意义下是单位, 故 $X \mapsto X^a$ 定义了一个合法的环自同构 $\sigma_a: \mathcal{R}_N^{\mathbb{R}} \rightarrow \mathcal{R}_N^{\mathbb{R}}$ 。并且 $\sigma_a(X^k) = -X^k$ 。

证明. 由于 $2n$ 为偶数, 所以 $a = 1 + 2n$ 为奇数。再由 $k = N/(2n)$ 可得 $2nk = N$ 。因此

$$\sigma_a(X^k) = X^{ka} = X^{k(1+2n)} = X^{k+2nk} = X^{k+N} = X^k \cdot X^N = X^k \cdot (-1) = -X^k.$$

据此定义 □

$$\text{ct}_{-} := \text{Auto}_{1+2n}(\text{ct}_P). \quad (1)$$

推论 3.7. 有 $\text{ct}_{-} \models_{\Delta} P(-X^k)$ 。

证明. 由 $\text{ct}_P \models_{\Delta} P(X^k)$ 以及命题 3.6, 再用自同构的消息语义即可:

$$\text{ct}_{-} = \text{Auto}_{1+2n}(\text{ct}_P) \models_{\Delta} P(\sigma_{1+2n}(X^k)) = P(-X^k).$$

□

第二步：得到偶分支 定义

$$\mathbf{ct}_{\text{even}}^{\uparrow} := \text{MulPt}_{\frac{1}{2}}(\text{Add}(\mathbf{ct}_P, \mathbf{ct}_{-})). \quad (2)$$

定理 3.8 (偶分支的正确性). 由 (2) 得到的密文满足

$$\mathbf{ct}_{\text{even}}^{\uparrow} \models_{\Delta} P_{\text{even}}(X^{2k}), \quad P_{\text{even}}(X^{2k}) = \sum_{i=0}^{n-1} p_{2i} X^{2ik}.$$

证明. $\text{Add}(\mathbf{ct}_P, \mathbf{ct}_{-}) \models_{\Delta} P(X^k) + P(-X^k)$, 再乘 $1/2$ 得 $\mathbf{ct}_{\text{even}}^{\uparrow} \models_{\Delta} \frac{P(X^k) + P(-X^k)}{2} = P_{\text{even}}((X^k)^2) = P_{\text{even}}(X^{2k})$. $\square$

第三步：得到带 Y 因子的奇分支 定义

$$\mathbf{ct}_{\text{odd}, Y}^{\uparrow} := \text{MulPt}_{\frac{1}{2}}(\text{Sub}(\mathbf{ct}_P, \mathbf{ct}_{-})). \quad (3)$$

定理 3.9 (带 Y 因子的奇分支的正确性). $\mathbf{ct}_{\text{odd}, Y}^{\uparrow} \models_{\Delta} X^k P_{\text{odd}}(X^{2k}) = \sum_{i=0}^{n-1} p_{2i+1} X^{(2i+1)k}$.

证明. $\text{Sub}(\mathbf{ct}_P, \mathbf{ct}_{-}) \models_{\Delta} P(X^k) - P(-X^k)$, 乘 $1/2$ 得 $\mathbf{ct}_{\text{odd}, Y}^{\uparrow} \models_{\Delta} \frac{P(X^k) - P(-X^k)}{2} = X^k P_{\text{odd}}(X^{2k})$. $\square$

第四步 (可选): 去掉奇分支前的 $Y = X^k$

引理 3.10 (求 $Y = X^k$ 的逆元). 在 $\mathcal{R}_N^{\mathbb{R}} = \mathbb{R}[X]/(X^N + 1)$ 中,

$$(X^k)^{-1} = -X^{N-k}.$$

证明. $X^k \cdot (-X^{N-k}) = -X^N = 1$. $\square$

定义

$$\mathbf{ct}_{\text{odd}}^{\uparrow} := \text{MulPt}_{-X^{N-k}}(\mathbf{ct}_{\text{odd}, Y}^{\uparrow}). \quad (4)$$

定理 3.11 (去掉 Y 因子的奇分支的正确性). $\mathbf{ct}_{\text{odd}}^{\uparrow} \models_{\Delta} P_{\text{odd}}(X^{2k}) = \sum_{i=0}^{n-1} p_{2i+1} X^{2ik}$.

证明. $\mathbf{ct}_{\text{odd}, Y}^{\uparrow} \models_{\Delta} X^k P_{\text{odd}}(X^{2k})$, 乘上 $-X^{N-k}$ 得 $\mathbf{ct}_{\text{odd}}^{\uparrow} \models_{\Delta} P_{\text{odd}}(X^{2k})$. $\square$

正向方向的最终结论 设输入密文 $\mathbf{ct}_P \models_{\Delta} P(X^k)$. 可通过以下密文操作得到两个分支:

- (1) $\mathbf{ct}_{-} := \text{Auto}_{1+2n}(\mathbf{ct}_P) \models_{\Delta} P(-X^k)$;
- (2) $\mathbf{ct}_{\text{even}}^{\uparrow} := \text{MulPt}_{\frac{1}{2}}(\text{Add}(\mathbf{ct}_P, \mathbf{ct}_{-})) \models_{\Delta} P_{\text{even}}(X^{2k})$;
- (3) $\mathbf{ct}_{\text{odd}, Y}^{\uparrow} := \text{MulPt}_{\frac{1}{2}}(\text{Sub}(\mathbf{ct}_P, \mathbf{ct}_{-})) \models_{\Delta} X^k P_{\text{odd}}(X^{2k})$;
- (4) 可选: $\mathbf{ct}_{\text{odd}}^{\uparrow} := \text{MulPt}_{-X^{N-k}}(\mathbf{ct}_{\text{odd}, Y}^{\uparrow}) \models_{\Delta} P_{\text{odd}}(X^{2k})$.

3.1.4 反向方向：由两个分支密文恢复原始密文

本节给出与上一节完全对应的逆过程。

情形 A：奇分支保留 Y 因子 假设手中已有两个密文：

$$\mathbf{ct}_{\text{even}}^{\uparrow} \models_{\Delta} P_{\text{even}}(X^{2k}), \quad \mathbf{ct}_{\text{odd},Y}^{\uparrow} \models_{\Delta} X^k P_{\text{odd}}(X^{2k}).$$

定义

$$\mathbf{ct}_P^{(A)} := \text{Add}(\mathbf{ct}_{\text{even}}^{\uparrow}, \mathbf{ct}_{\text{odd},Y}^{\uparrow}). \quad (5)$$

定理 3.12 (情形 A 的正确性). $\mathbf{ct}_P^{(A)} \models_{\Delta} P(X^k)$ 。

证明. 由加法的消息语义：

$$\mathbf{ct}_P^{(A)} \models_{\Delta} P_{\text{even}}(X^{2k}) + X^k P_{\text{odd}}(X^{2k}) = P(X^k) \quad .$$

□

情形 B：奇分支已去掉 Y 因子 假设已有

$$\mathbf{ct}_{\text{even}}^{\uparrow} \models_{\Delta} P_{\text{even}}(X^{2k}), \quad \mathbf{ct}_{\text{odd}}^{\uparrow} \models_{\Delta} P_{\text{odd}}(X^{2k}).$$

为了恢复 $P(X^k)$ ，先将奇分支乘回 X^k ：

$$\mathbf{ct}_P^{(B)} := \text{Add}\left(\mathbf{ct}_{\text{even}}^{\uparrow}, \text{MulPt}_{X^k}(\mathbf{ct}_{\text{odd}}^{\uparrow})\right). \quad (6)$$

定理 3.13 (情形 B 的正确性). $\mathbf{ct}_P^{(B)} \models_{\Delta} P(X^k)$ 。

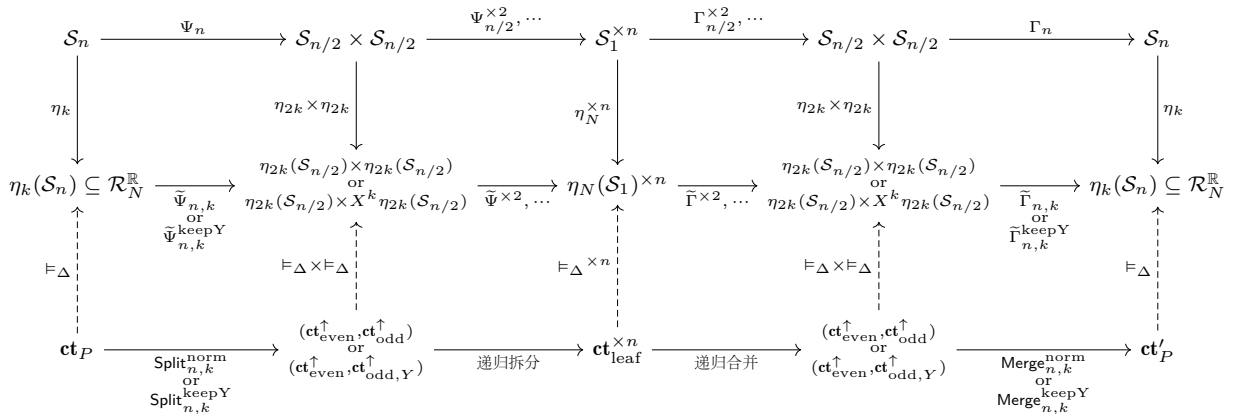
证明. $\text{MulPt}_{X^k}(\mathbf{ct}_{\text{odd}}^{\uparrow}) \models_{\Delta} X^k P_{\text{odd}}(X^{2k})$ ，加上 $\mathbf{ct}_{\text{even}}^{\uparrow}$ 得 $\mathbf{ct}_P^{(B)} \models_{\Delta} P_{\text{even}}(X^{2k}) + X^k P_{\text{odd}}(X^{2k}) = P(X^k)$ 。 □

反向方向的最终结论 设 $\mathbf{ct}_{\text{even}}^{\uparrow}, \mathbf{ct}_{\text{odd},Y}^{\uparrow}, \mathbf{ct}_{\text{odd}}^{\uparrow}$ 分别满足

$$\mathbf{ct}_{\text{even}}^{\uparrow} \models_{\Delta} P_{\text{even}}(X^{2k}), \quad \mathbf{ct}_{\text{odd},Y}^{\uparrow} \models_{\Delta} X^k P_{\text{odd}}(X^{2k}), \quad \mathbf{ct}_{\text{odd}}^{\uparrow} \models_{\Delta} P_{\text{odd}}(X^{2k}).$$

则有两种重构方式：

- (1) 若奇分支保留 X^k ，则 $\mathbf{ct}_P^{(A)} := \text{Add}(\mathbf{ct}_{\text{even}}^{\uparrow}, \mathbf{ct}_{\text{odd},Y}^{\uparrow}) \models_{\Delta} P(X^k)$ ；
- (2) 若奇分支已去掉 X^k ，则 $\mathbf{ct}_P^{(B)} := \text{Add}(\mathbf{ct}_{\text{even}}^{\uparrow}, \text{MulPt}_{X^k}(\mathbf{ct}_{\text{odd}}^{\uparrow})) \models_{\Delta} P(X^k)$ 。



3.2 大环与小环之间的密文转换

本节讨论两个互相关联的问题。第一个是**压缩**：当一个大环密文的明文具有子环形式 $m(X^2)$ 时，如何把它转换成小环密文，使其明文变为 $m(Y)$ 。第二个是**合并**：如果之后需要恢复父层大环密文，如何把小环密文升回大环，并按奇偶重构公式合并。

3.2.1 大环到小环密文压缩

问题设置与基本符号. 设 n 是 2 的幂，令 $N = 2n$ ，并定义小环和大环

$$R_n = \mathbb{Z}_q[Y]/(Y^n + 1), \quad R_N = \mathbb{Z}_q[X]/(X^N + 1).$$

通过 $Y = X^2$ 可以把 R_n 嵌入到 R_N 中，因为在大环中有 $(X^2)^n = X^{2n} = X^N = -1$ 。

本问题的输入是一个大环密文 $\mathbf{ct} = (c_0(X), c_1(X)) \in R_N^2$ ，它在大环私钥 $s(X) \in R_N$ 下满足

$$c_0(X) + s(X)c_1(X) = \Delta m(X^2) + e(X) \pmod{q}.$$

其中 $m(Y) \in R_n$ 是目标小环明文。我们的目标是构造一个小环密文 $\mathbf{ct}' = (d_0(Y), d_1(Y)) \in R_n^2$ ，使其在目标小环私钥 $s'(Y) \in R_n$ 下满足

$$d_0(Y) + s'(Y)d_1(Y) = \Delta m(Y) + e'(Y) \pmod{q}.$$

公开线性映射 τ . 定义公开线性映射 $\tau : R_N \rightarrow R_n$ 。若 $F(X) = \sum_{j=0}^{2n-1} f_j X^j$ ，则

$$\tau(F)(Y) = \sum_{i=0}^{n-1} f_{2i} Y^i.$$

也就是说， τ 提取偶次项系数，并把 X^{2i} 重新编号为 Y^i 。注意， τ 是 $\mathbb{Z}_q$ -线性映射，但不是环同态。例如：若 $c_i = a_0 + a_1 X + a_2 X^2 + a_3 X^3$ ，则 $\tau(c_i) = a_0 + a_2 Y$ ，而 $\tau(X^{-1}c_i) = a_1 + a_3 Y$ ；其中 X^{-1} 的作用是把奇次项 X^1, X^3 移到偶次位置 X^0, X^2 ，从而再经 τ 提取出原多项式的奇部。

由于 $X^N = X^{2n} = -1$ ，在 R_N 中有 $X^{-1} = -X^{2n-1}$ 。因此乘以 X^{-1} 是一个公开的单项式移位操作。

公开提取小环多项式. 因为密文分量 c_0, c_1 是公开多项式，所以可以直接计算

$$A = \tau(c_0), B = \tau(c_1), C = \tau(X^{-1}c_0), D = \tau(X^{-1}c_1).$$

其中 $A, B, C, D \in R_n$ 。直观上， A, B 是 c_0, c_1 的偶部；而 C, D 是先把奇次项通过乘 X^{-1} 移到偶次位置，再用 τ 提取出来，所以对应 c_0, c_1 的奇部。

更具体地，若 $F(X) = \sum_{i=0}^{n-1} f_{2i} X^{2i} + \sum_{i=0}^{n-1} f_{2i+1} X^{2i+1}$ ，则

$$\tau(F) = \sum_{i=0}^{n-1} f_{2i} Y^i, \quad \tau(X^{-1}F) = \sum_{i=0}^{n-1} f_{2i+1} Y^i.$$

因此 $\tau(F)$ 提取偶部，而 $\tau(X^{-1}F)$ 提取奇部。

重写大环密文分量。 对任意 $F(X) \in R_N$ ，都有分解

$$F(X) = \tau(F)(X^2) + X \cdot \tau(X^{-1}F)(X^2).$$

这是因为第一项恢复偶次项，第二项恢复奇次项。

因此，对 c_0, c_1 有

$$c_0(X) = A(X^2) + XC(X^2), \quad c_1(X) = B(X^2) + XD(X^2).$$

分解大环私钥。 同样地，大环私钥可以唯一写成偶部和奇部：

$$s(X) = s_0(X^2) + Xs_1(X^2), \quad s_0(Y), s_1(Y) \in R_n.$$

这里 s_0 和 s_1 分别是大环私钥的偶部和奇部。实际算法不需要知道 s_0, s_1 ，它们只出现在正确性证明和密钥切换材料中。

代入解密方程。 为了避免混淆小环多项式和它们在大环中的嵌入，记

$$\tilde{A} = A(X^2), \quad \tilde{B} = B(X^2), \quad \tilde{C} = C(X^2), \quad \tilde{D} = D(X^2),$$

并同样记

$$\tilde{s}_0 = s_0(X^2), \quad \tilde{s}_1 = s_1(X^2).$$

于是

$$c_0(X) = \tilde{A} + X\tilde{C}, \quad c_1(X) = \tilde{B} + X\tilde{D}, \quad s(X) = \tilde{s}_0 + X\tilde{s}_1.$$

代入原始解密方程，得到

$$c_0 + sc_1 = \tilde{A} + X\tilde{C} + (\tilde{s}_0 + X\tilde{s}_1)(\tilde{B} + X\tilde{D}).$$

展开为

$$c_0 + sc_1 = \tilde{A} + X\tilde{C} + \tilde{s}_0\tilde{B} + X\tilde{s}_0\tilde{D} + X\tilde{s}_1\tilde{B} + X^2\tilde{s}_1\tilde{D}.$$

因此偶次部分为

$$\tilde{A} + \tilde{s}_0\tilde{B} + X^2\tilde{s}_1\tilde{D},$$

奇次部分为

$$X(\tilde{C} + \tilde{s}_0\tilde{D} + \tilde{s}_1\tilde{B}).$$

另一方面，右边 $\Delta m(X^2)$ 只含偶次项，而噪声可以唯一分解为

$$e(X) = e_{\text{even}}(X^2) + Xe_{\text{odd}}(X^2).$$

因此比较奇偶部分得到

$$\tilde{A} + \tilde{s}_0\tilde{B} + X^2\tilde{s}_1\tilde{D} = \Delta m(X^2) + e_{\text{even}}(X^2),$$

以及

$$\tilde{C} + \tilde{s}_0\tilde{D} + \tilde{s}_1\tilde{B} = e_{\text{odd}}(X^2).$$

得到小环方程。 对上述两个等式应用“把 X^2 重新记为 Y ”的反嵌入，得到小环中的方程：

$$A + s_0 B + Y(s_1 D) = \Delta m(Y) + \nu_0,$$

$$C + s_0 D + s_1 B = \nu_1.$$

其中 ν_0, ν_1 分别来自原始噪声的偶部和奇部。第一个方程携带目标消息 $m(Y)$ ，第二个方程近似加密零。

构造秩二模块密文。 将两个小环方程写成矩阵形式。定义

$$\mathbf{t} = \begin{pmatrix} A \\ C \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} B & YD \\ D & B \end{pmatrix}, \quad \mathbf{s} = \begin{pmatrix} s_0 \\ s_1 \end{pmatrix}.$$

则有

$$\mathbf{t} + \mathbf{U}\mathbf{s} = \begin{pmatrix} A + s_0 B + Y(s_1 D) \\ C + s_0 D + s_1 B \end{pmatrix} = \begin{pmatrix} \Delta m(Y) + \nu_0 \\ \nu_1 \end{pmatrix}.$$

因此， $(\mathbf{t}, \mathbf{U})$ 可以看作一个定义在小环 R_n 上、绑定向量私钥 $\mathbf{s}$ 的秩二模块密文。它的第一行携带目标消息，第二行近似为零。

密钥切换为普通小环密文。 上一步得到的是一个小环上的秩二模块密文。它的第一行满足

$$A + s_0 B + Y(s_1 D) = \Delta m(Y) + \nu_0.$$

其中私钥不是普通 CKKS 中的一个标量私钥，而是向量私钥 (s_0, s_1) 。普通小环密文希望具有形式

$$(d_0, d_1) \in R_n^2,$$

并且在目标小环私钥 $s'(Y) \in R_n$ 下满足

$$d_0 + s' d_1 \approx \Delta m(Y).$$

因此这里需要做一次从向量私钥 (s_0, s_1) 到标量私钥 s' 的 gadget key switching。

为了说明这个过程，先考虑一般情形。设有三元素形式 $(t, u, v) \in R_n^3$ ，并且存在旧噪声 η_{old} ，使得

$$t + s_0 u + s_1 v = \text{msg} + \eta_{\text{old}}.$$

我们的目标是把它转换成普通二元素密文 (d_0, d_1) ，使得

$$d_0 + s' d_1 = \text{msg} + \eta_{\text{old}} + \eta_{\text{ks}}.$$

选择 gadget 基 G 和分解长度 L 。对 u 和 v 做 gadget 分解：

$$u = \sum_{\ell=0}^{L-1} u_\ell G^\ell, \quad v = \sum_{\ell=0}^{L-1} v_\ell G^\ell.$$

这里 $u_\ell, v_\ell \in R_n$ 是分解后的 digit 多项式，通常要求其系数较小。

接下来生成两组切换密钥，分别用于处理 s_0u 和 s_1v 。为了和本文一直采用的解密约定

$$d_0 + s'd_1$$

保持一致，切换密钥应定义为

$$b_{0,\ell} = G^\ell s_0 - a_{0,\ell} s' + \epsilon_{0,\ell}, \quad b_{1,\ell} = G^\ell s_1 - a_{1,\ell} s' + \epsilon_{1,\ell},$$

即：

$$b_{0,\ell} + a_{0,\ell} s' = G^\ell s_0 + \epsilon_{0,\ell}, \quad b_{1,\ell} + a_{1,\ell} s' = G^\ell s_1 + \epsilon_{1,\ell},$$

其中 $a_{0,\ell}, a_{1,\ell} \in R_n$ 是均匀随机多项式， $\epsilon_{0,\ell}, \epsilon_{1,\ell}$ 是小噪声。因此公开的切换密钥为：

$$\text{KSK}_0 = \{(b_{0,\ell}, a_{0,\ell})\}_{\ell=0}^{L-1}, \quad \text{KSK}_1 = \{(b_{1,\ell}, a_{1,\ell})\}_{\ell=0}^{L-1}.$$

换句话说， $(b_{0,\ell}, a_{0,\ell})$ 、 $(b_{1,\ell}, a_{1,\ell})$ 是在新私钥 s' 下，对 $G^\ell s_0$ 、 $G^\ell s_1$ 的 CKKS 密文。只是它们加密的不是用户消息，而是旧私钥分量的 gadget 倍数。

使用这些切换密钥时，定义

$$d_0 = t + \sum_{\ell=0}^{L-1} u_\ell b_{0,\ell} + \sum_{\ell=0}^{L-1} v_\ell b_{1,\ell},$$

$$d_1 = \sum_{\ell=0}^{L-1} u_\ell a_{0,\ell} + \sum_{\ell=0}^{L-1} v_\ell a_{1,\ell}.$$

下面验证输出确实绑定到目标私钥 s' 。由定义可得

$$\begin{aligned} d_0 + s'd_1 &= t + \sum_{\ell} u_\ell b_{0,\ell} + \sum_{\ell} v_\ell b_{1,\ell} + s' \left(\sum_{\ell} u_\ell a_{0,\ell} + \sum_{\ell} v_\ell a_{1,\ell} \right) \\ &= t + \sum_{\ell} u_\ell (b_{0,\ell} + s'a_{0,\ell}) + \sum_{\ell} v_\ell (b_{1,\ell} + s'a_{1,\ell}). \end{aligned}$$

由于环 R_n 是交换环， $s'a_{i,\ell} = a_{i,\ell}s'$ 。根据切换密钥定义，

$$b_{0,\ell} + s'a_{0,\ell} = G^\ell s_0 + \epsilon_{0,\ell}, \quad b_{1,\ell} + s'a_{1,\ell} = G^\ell s_1 + \epsilon_{1,\ell}.$$

因此

$$\begin{aligned} d_0 + s'd_1 &= t + \sum_{\ell} u_\ell (G^\ell s_0 + \epsilon_{0,\ell}) + \sum_{\ell} v_\ell (G^\ell s_1 + \epsilon_{1,\ell}) \\ &= t + s_0 \sum_{\ell} u_\ell G^\ell + s_1 \sum_{\ell} v_\ell G^\ell + \sum_{\ell} u_\ell \epsilon_{0,\ell} + \sum_{\ell} v_\ell \epsilon_{1,\ell} \\ &= t + s_0 u + s_1 v + \eta_{\text{ks}}, \end{aligned}$$

其中

$$\eta_{\text{ks}} = \sum_{\ell} u_\ell \epsilon_{0,\ell} + \sum_{\ell} v_\ell \epsilon_{1,\ell}$$

是 key switching 引入的新噪声。于是如果原来有

$$t + s_0 u + s_1 v = \text{msg} + \eta_{\text{old}},$$

那么输出密文满足

$$d_0 + s'd_1 = \text{msg} + \eta_{\text{old}} + \eta_{\text{ks}}.$$

在本问题中，模块密文的第一行对应三元素密文

$$(t, u, v) = (A, B, YD),$$

因为

$$A + s_0B + Y(s_1D) = \Delta m(Y) + \nu_0.$$

对这一行执行上述 key switching，得到普通小环密文

$$\mathbf{ct}' = (d_0, d_1) \in R_n^2,$$

并且它满足

$$d_0 + s'd_1 = \Delta m(Y) + \nu_0 + \eta_{\text{ks}}.$$

因此 $\mathbf{ct}'$ 就是目标输出小环密文。

第二行对应三元素密文

$$(t, u, v) = (C, D, B),$$

因为

$$C + s_0D + s_1B = \nu_1.$$

它近似加密零。通常最终输出只需要第一行切换得到的密文；第二行主要用于说明原始解密关系在小环中的完整结构。若额外对第二行执行 key switching，则会得到一个近似加密零的普通小环密文，但把它加到输出中一般只会增加额外噪声，因此默认不使用。

压缩正确性总结。 整个压缩过程从大环解密关系

$$c_0 + sc_1 = \Delta m(X^2) + e$$

出发。通过 τ 和 X^{-1} 可以公开提取密文分量 c_0, c_1 的偶部和奇部，得到 A, B, C, D 。再将大环私钥分解为 $s = s_0(X^2) + Xs_1(X^2)$ ，原解密关系的偶部可以转化为小环方程

$$A + s_0B + Y(s_1D) = \Delta m(Y) + \nu_0.$$

这正是一个绑定向量私钥 (s_0, s_1) 的三元素密文方程。最后通过 gadget key switching，将该三元素密文从向量私钥 (s_0, s_1) 切换到目标小环标量私钥 s' ，得到普通小环密文 (d_0, d_1) ，满足

$$d_0 + s'd_1 = \Delta m(Y) + \nu_0 + \eta_{\text{ks}}.$$

因此，最终输出密文解密后得到的就是目标小环消息 $m(Y)$ ，只差 CKKS 缩放因子和总噪声项。

3.2.2 压缩后的合并：小环到大环与父层重构

什么时候需要合并。 压缩之后不一定需要马上合并。如果后续算法继续在小环中递归计算，例如继续对 $P_{\text{even}}(Y)$ 或 $P_{\text{odd}}(Y)$ 做下一层拆分，那么合并回大环没有必要，反而会增加成本。

只有当最终需要恢复父层的大环密文时，才需要执行合并。此时合并分成两件事：先把小环密文升回大环，再按奇偶重构公式合并。

未压缩时的重构。 如果只做了奇偶拆分，但没有做大环到小环压缩，那么两个分支仍然是大环密文。假设有

$$\mathbf{ct}_{\text{even}} \text{ encrypts } P_{\text{even}}(X^2), \quad \mathbf{ct}_{\text{odd}} \text{ encrypts } P_{\text{odd}}(X^2).$$

由于明文层有 $P(X) = P_{\text{even}}(X^2) + XP_{\text{odd}}(X^2)$ ，所以直接计算

$$\mathbf{ct}_P = \mathbf{ct}_{\text{even}} + X \cdot \mathbf{ct}_{\text{odd}}.$$

如果奇分支本身已经加密 $XP_{\text{odd}}(X^2)$ ，则只需 $\mathbf{ct}_P = \mathbf{ct}_{\text{even}} + \mathbf{ct}_{\text{odd},X}$ 。

压缩后为什么不能直接合并。 如果两个分支已经压缩到小环，例如

$$\mathbf{ct}'_e \text{ encrypts } P_{\text{even}}(Y), \quad \mathbf{ct}'_o \text{ encrypts } P_{\text{odd}}(Y),$$

那么它们生活在 R_n 中，而父层多项式 $P(X)$ 生活在 R_N 中。因此不能直接写 $\mathbf{ct}'_e + X\mathbf{ct}'_o$ ，因为 $\mathbf{ct}'_e$ 和 $\mathbf{ct}'_o$ 中没有大环变量 X 。必须先做变量嵌入 $Y \mapsto X^2$ ，把两个小环密文升回大环。

小环到大环的升环。 设小环密文 $\mathbf{ct}' = (d_0(Y), d_1(Y))$ 满足

$$d_0(Y) + s'(Y)d_1(Y) \approx \Delta f(Y).$$

先对密文分量做公开嵌入，把 Y^i 替换为 X^{2i} ，得到 $d_0(X^2)$ 和 $d_1(X^2)$ 。例如 $a_0 + a_1Y + a_2Y^2$ 会被嵌入为 $a_0 + a_1X^2 + a_2X^4$ 。数组上看，就是在系数之间插入零。

嵌入后得到大环关系

$$d_0(X^2) + s'(X^2)d_1(X^2) \approx \Delta f(X^2).$$

此时它已经是一个大环密文语义，但绑定的私钥是 $s'(X^2)$ ，不一定是目标大环私钥 $S(X)$ 。

切换到目标大环私钥。 如果后续计算允许使用 $s'(X^2)$ 作为大环私钥，那么升环到这里已经足够。但如果希望回到原来的大环私钥 $S(X)$ ，还需要执行一次 key switching:

$$s'(X^2) \longrightarrow S(X).$$

这一步是小环到大环合并过程中真正昂贵的部分。公开插零嵌入本身很便宜，主要成本来自这次 key switching。

具体地，设升环后的两个大环多项式为

$$D_0(X) = d_0(X^2), \quad D_1(X) = d_1(X^2).$$

此时有

$$D_0(X) + s'(X^2)D_1(X) \approx \Delta f(X^2).$$

若希望切换到目标大环私钥 $S(X)$ ，可以生成 expansion key

$$\beta_\ell = G^\ell s'(X^2) - \alpha_\ell S(X) + \epsilon_\ell, \quad \ell = 0, \dots, L-1.$$

将

$$D_1 = \sum_{\ell=0}^{L-1} D_{1,\ell} G^\ell$$

作 gadget 分解, 并定义

$$C_0 = D_0 + \sum_{\ell=0}^{L-1} D_{1,\ell} \beta_\ell, \quad C_1 = \sum_{\ell=0}^{L-1} D_{1,\ell} \alpha_\ell.$$

则

$$C_0 + SC_1 = D_0 + s'(X^2)D_1 + \text{key switching noise} \approx \Delta f(X^2).$$

因此 (C_0, C_1) 就是绑定目标大环私钥 $S(X)$ 的大环密文。

父层重构公式. 假设偶分支和奇分支升环后得到大环密文 $\mathbf{Ct}_e$ 和 $\mathbf{Ct}_o$, 分别加密

$$P_{\text{even}}(X^2), \quad P_{\text{odd}}(X^2).$$

则父层重构为

$$\mathbf{Ct}_P = \mathbf{Ct}_e + X \cdot \mathbf{Ct}_o.$$

于是 $\mathbf{Ct}_P$ 加密

$$P_{\text{even}}(X^2) + X P_{\text{odd}}(X^2) = P(X).$$

如果奇分支保留了 X 因子, 即它加密的是 $X P_{\text{odd}}(X^2)$, 则重构公式简化为 $\mathbf{Ct}_P = \mathbf{Ct}_e + \mathbf{Ct}_{o,X}$ 。

合并不是压缩的严格逆操作. 需要强调的是, 合并回去通常不能恢复原始密文本身。压缩过程只保留了用于恢复目标消息的偶部方程, 并通过 key switching 重新生成了一个新的小环密文。因此它不是原始大环密文 (c_0, c_1) 的可逆变换。

但是这在同态加密中是可以接受的。我们关心的是密文语义, 即解密后是否得到同一个明文。升环并合并后得到的是一个新的大环密文, 它一般不等于原密文, 但可以解密到同一个 $P(X)$ 。

噪声与复杂度. 若两个小环分支的噪声分别为 $e_e(Y)$ 和 $e_o(Y)$, 升环后变为 $e_e(X^2)$ 和 $e_o(X^2)$ 。合并后噪声大致为

$$e_{\text{merge}}(X) = e_e(X^2) + X e_o(X^2) + e_{\text{expandKS}}(X),$$

其中 e_{expandKS} 来自升环后切换到目标大环私钥的 key switching。如果不需要切换私钥, 则没有这部分噪声。

复杂度方面, 插零嵌入、乘 X 、加法都只是 $O(N)$ 的公开或线性操作。真正的主要成本是从 $s'(X^2)$ 到 $S(X)$ 的大环 key switching。如果偶、奇两个分支都要升环并切换回同一个目标大环私钥, 通常需要两次这样的 key switching。

注 3.14. 如果不合并切换大环私钥的优点是省时间省操作, 缺点是不可和之前在原先大环私钥的 $S(X)$ 下的密文做同态运算。

4 复杂度与噪声分析

本节分析前文两个核心操作的噪声增长、层数消耗和主要复杂度。为避免实现细节干扰主线，本文采用如下抽象模型。所有噪声均在解密后的明文多项式层面讨论，即若密文 $\mathbf{ct} = (c_0, c_1)$ 在私钥 s 下满足 $\text{Dec}_s(\mathbf{ct}) = c_0 + sc_1 = \Delta m + e \pmod{q}$ ，则称 e 为当前密文噪声。除特别说明外，本节使用系数无穷范数 $\|\cdot\|_\infty$ 。在该范数下，自同构 $X \mapsto X^a$ 和乘以单项式只造成系数置换与符号变化，因此不改变噪声范数；而在 $R_n = \mathbb{Z}_q[Y]/(Y^n + 1)$ 中，有保守卷积上界 $\|fg\|_\infty \leq n\|f\|_\infty\|g\|_\infty$ 。

本节只刻画主要噪声项。具体实现中的 RNS basis conversion、mod-down、rescale rounding、scale matching 以及 plaintext encoding rounding 可能引入额外误差。若实现中执行了这些步骤，应将相应误差另行加入本文给出的主噪声表达式中。

4.1 FFT 式奇偶拆分的噪声

先考虑从 $\mathbf{ct}_P$ 同态得到偶分支和奇分支的过程。设输入密文满足 $\text{Dec}_s(\mathbf{ct}_P) = \Delta P(X^k) + e_P(X)$ ，且 $\|e_P\|_\infty \leq E_P$ 。第一步计算 $\mathbf{ct}_- = \text{Auto}_{1+2n}(\mathbf{ct}_P)$ 。该自同构将消息从 $P(X^k)$ 变为 $P(-X^k)$ ，同时由于包含 Galois key switching，会引入一项新增噪声 η_{auto} 。因此可写为

$$\text{Dec}_s(\mathbf{ct}_-) = \Delta P(-X^k) + \sigma_{1+2n}(e_P) + \eta_{\text{auto}}.$$

其中 σ_{1+2n} 是对应的噪声自同构。由于它在系数层面只是置换与符号变化，有 $\|\sigma_{1+2n}(e_P)\|_\infty = \|e_P\|_\infty$ 。

偶分支定义为 $\mathbf{ct}_{\text{even}}^\uparrow = \text{MulPt}_{1/2}(\mathbf{ct}_P + \mathbf{ct}_-)$ ，带 Y 因子的奇分支定义为 $\mathbf{ct}_{\text{odd},Y}^\uparrow = \text{MulPt}_{1/2}(\mathbf{ct}_P - \mathbf{ct}_-)$ 。在忽略常数明文乘法的额外舍入误差时，两者噪声分别为

$$e_{\text{even}}^\uparrow = \frac{e_P + \sigma_{1+2n}(e_P) + \eta_{\text{auto}}}{2}, \quad e_{\text{odd},Y}^\uparrow = \frac{e_P - \sigma_{1+2n}(e_P) - \eta_{\text{auto}}}{2}.$$

若记 $\|\eta_{\text{auto}}\|_\infty \leq E_{\text{auto}}$ ，则两个分支均满足保守上界

$$\|e_{\text{even}}^\uparrow\|_\infty, \|e_{\text{odd},Y}^\uparrow\|_\infty \leq E_P + \frac{1}{2}E_{\text{auto}}.$$

如果进一步计算归一化奇分支 $\mathbf{ct}_{\text{odd}}^\uparrow = \text{MulPt}_{-X^{N-k}}(\mathbf{ct}_{\text{odd},Y}^\uparrow)$ ，由于 $-X^{N-k}$ 是单项式，乘法只导致系数循环移位和符号变化，因此 $\|e_{\text{odd}}^\uparrow\|_\infty = \|e_{\text{odd},Y}^\uparrow\|_\infty$ 。

因此，FFT 式奇偶拆分的主要新增噪声来自一次自同构 key switching。加法、减法、乘以 $1/2$ 和乘以单项式在本文抽象模型中不引入新的 key switching 噪声；若 $1/2$ 的 plaintext 表示或 scale matching 在具体实现中产生舍入误差，则应额外加入一个常数明文乘法误差项。

4.2 大环到小环压缩的噪声

接下来分析大环到小环压缩。输入大环密文满足

$$c_0(X) + s(X)c_1(X) = \Delta m(X^2) + e(X).$$

由于目标消息 $\Delta m(X^2)$ 只含 X 的偶次幂，噪声可唯一写成 $e(X) = e_0(X^2) + Xe_1(X^2)$ 。前文推导得到两个小环方程：第一行 $A + s_0B + Y(s_1D) = \Delta m(Y) + \nu_0$ ，第二行 $C + s_0D + s_1B = \nu_1$ ，其中 $\nu_0 = e_0(Y)$ ， $\nu_1 = e_1(Y)$ 。因此，第一行携带目标消息，第二行近似加密零。

压缩默认只使用第一行。令 $t = A$ 、 $u = B$ 、 $v = YD$ ，则第一行可写成 $t + s_0u + s_1v = \Delta m(Y) + \nu_0$ 。它不是普通二元 CKKS 密文，因为解密表达式中出现的是向量私钥 (s_0, s_1) ，而不是一个标量私钥 s' 。因此需要通过 gadget key switching 将其转换为 s' 下的普通小环密文。

设 gadget 基为 G ，分解长度为 L ，并写 $u = \sum_{\ell=0}^{L-1} u_\ell G^\ell$ 、 $v = \sum_{\ell=0}^{L-1} v_\ell G^\ell$ 。切换密钥由两组 RLWE 型样本组成：

$$b_{0,\ell} + s' a_{0,\ell} = G^\ell s_0 + \epsilon_{0,\ell}, \quad b_{1,\ell} + s' a_{1,\ell} = G^\ell s_1 + \epsilon_{1,\ell}.$$

使用这些 key 后，输出密文 (d_0, d_1) 满足

$$d_0 + s' d_1 = t + s_0 u + s_1 v + \eta_{\text{ks}}, \quad \eta_{\text{ks}} = \sum_{\ell=0}^{L-1} u_\ell \epsilon_{0,\ell} + \sum_{\ell=0}^{L-1} v_\ell \epsilon_{1,\ell}.$$

在本算法中 $u = B$ 、 $v = YD$ ，所以 $\eta_{\text{ks}} = \sum_{\ell} B_\ell \epsilon_{0,\ell} + \sum_{\ell} (YD)_\ell \epsilon_{1,\ell}$ 。因此最终输出小环密文满足

$$d_0 + s' d_1 = \Delta m(Y) + \nu_0 + \eta_{\text{ks}}.$$

也就是说，压缩输出噪声为 $e_{\text{out}} = \nu_0 + \eta_{\text{ks}}$ 。这个表达式揭示了压缩操作的两个效果：它继承原始噪声的偶部 ν_0 ，但会额外加入一次模块到环的 key switching 噪声 η_{ks} 。原始噪声的奇部 ν_1 不进入默认输出。

4.3 压缩噪声上界

为了得到一个保守估计，假设 $\|\nu_0\|_\infty \leq E_0$ ，切换密钥误差满足 $\|\epsilon_{0,\ell}\|_\infty, \|\epsilon_{1,\ell}\|_\infty \leq E_{\text{ks}}$ ，并且 gadget digit 满足 $\|u_\ell\|_\infty, \|v_\ell\|_\infty \leq B_{\text{dig}}$ 。由 $\|fg\|_\infty \leq n\|f\|_\infty\|g\|_\infty$ 可得

$$\|\eta_{\text{ks}}\|_\infty \leq \sum_{\ell} \|u_\ell \epsilon_{0,\ell}\|_\infty + \sum_{\ell} \|v_\ell \epsilon_{1,\ell}\|_\infty \leq 2Ln B_{\text{dig}} E_{\text{ks}}.$$

因此，输出噪声满足保守上界 $\|e_{\text{out}}\|_\infty \leq E_0 + 2Ln B_{\text{dig}} E_{\text{ks}}$ 。该上界通常较松，因为它使用了最坏情形的系数卷积估计。实际实现中可以根据 RNS 表示、NTT 乘法、digit 分布和误差统计给出更精细的平均情形估计。但从渐近和结构上看，压缩操作的主要新增噪声仍然来自 key switching。

4.4 第二行为什么默认不加入输出

第二行满足 $C + s_0 D + s_1 B = \nu_1$ ，因此它对应一个近似加密零的三元素密文。如果也对第二行执行 key switching，则会得到一个普通小环零密文，其噪声大致为 $\nu_1 + \eta_{\text{ks},1}$ 。把它加到第一行输出中不会改变消息，但会把噪声从 $\nu_0 + \eta_{\text{ks},0}$ 变为 $\nu_0 + \eta_{\text{ks},0} + \nu_1 + \eta_{\text{ks},1}$ 。除非有额外的噪声抵消结构，否则这通常只会增加误差。因此默认压缩算法只使用第一行 (A, B, YD) ，而不把第二行加入最终输出。

4.5 压缩是否具有降噪效果

压缩确实有一种投影效果：原始噪声 $e(X) = e_0(X^2) + X e_1(X^2)$ 中的奇部 e_1 不进入默认输出，输出只继承偶部 $\nu_0 = e_0(Y)$ 。但是这并不意味着压缩一定降噪，因为同时新增了 key

switching 噪声 η_{ks} 。因此应比较的是 $\|\nu_0 + \eta_{ks}\|$ 与 $\|e_0(X^2) + Xe_1(X^2)\|$ ，而不是只比较 $\|\nu_0\|$ 与 $\|e\|$ 。

更准确地说，压缩是“噪声投影加 key switching 噪声注入”。当原始噪声奇部较大且 key switching 噪声较小时，压缩后噪声可能更小；当原始噪声本来较小或 key switching 噪声较大时，压缩后噪声可能反而更大。因此本文只声称压缩改变噪声结构，不声称其必然具有降噪效果。

4.6 压缩后合并的噪声

如果后续计算继续留在小环中，则不需要合并。只有当需要恢复父层大环密文时，才需要先执行变量嵌入 $Y \mapsto X^2$ ，再按奇偶公式重构。设两个小环分支分别满足 $\mathbf{ct}'_e$ 加密 $P_{\text{even}}(Y)$ ，噪声为 $e_e(Y)$ ； $\mathbf{ct}'_o$ 加密 $P_{\text{odd}}(Y)$ ，噪声为 $e_o(Y)$ 。公开嵌入后，两个噪声分别变为 $e_e(X^2)$ 和 $e_o(X^2)$ 。

如果合并后允许继续使用嵌入私钥 $s'(X^2)$ ，则可以直接令 $F_0 = E_0 + XO_0$ 、 $F_1 = E_1 + XO_1$ ，得到 $F_0 + s'(X^2)F_1 \approx \Delta(P_{\text{even}}(X^2) + XP_{\text{odd}}(X^2))$ 。此时合并噪声为 $e_e(X^2) + Xe_o(X^2)$ ，没有额外的升环 key switching 噪声。

如果要求输出回到目标大环私钥 $S(X)$ 下，则还需要执行一次从 $s'(X^2)$ 到 $S(X)$ 的 key switching。若将该步骤引入的噪声记为 η_{exp} ，则先合并后切换的输出噪声可写为 $e_{\text{merge}}(X) = e_e(X^2) + Xe_o(X^2) + \eta_{\text{exp}}(X)$ 。如果实现选择先分别对偶、奇两个分支做升环 key switching，再合并，则噪声可写为 $e_e(X^2) + Xe_o(X^2) + \eta_{\text{exp},e}(X) + X\eta_{\text{exp},o}(X)$ 。前一种方式在两个分支绑定同一嵌入私钥且 level、scale 一致时通常更省 key switching。

4.7 层数消耗

本文采用的抽象口径是：只有 rescale 或显式 modulus switching 会消耗 CKKS level；加法、减法、自同构、key switching、乘以单项式、公开重排和 gadget 分解本身不主动消耗 level，但会增加噪声或需要 evaluation key。在此口径下，FFT 拆分中的 $\mathbf{Auto}_{1+2n}$ 、加减法、乘以 $1/2$ 、乘以 $-X^{N-k}$ ，以及压缩中的 τ 、 X^{-1} 、构造 YD 、gadget decomposition 和 key switching，默认 level 消耗均为 0。小环升回大环时，公开嵌入 $Y \mapsto X^2$ 也不消耗 level；若进一步执行 $s'(X^2) \rightarrow S(X)$ 的 key switching，通常同样不主动消耗 level。

因此，在不执行额外 rescale 或 mod-down 的抽象模型中，单次“拆分 + 压缩”的 level 消耗可以记为 0 层。但是这并不表示该操作免费：它会引入自同构 key switching 噪声和模块压缩 key switching 噪声。具体实现若为了 scale matching、RNS basis management 或主动降模而执行 rescale/mod-down，则应按实际次数计入 level 消耗。

4.8 复杂度分析

设 $N = 2n$ 。大环到小环压缩主要包含四部分。第一，公开计算 $A = \tau(c_0)$ 、 $B = \tau(c_1)$ 、 $C = \tau(X^{-1}c_0)$ 、 $D = \tau(X^{-1}c_1)$ ，这只需要扫描 c_0, c_1 的系数，复杂度为 $O(N)$ 。第二，在小环中计算 YD ，这是单项式移位和符号变化，复杂度为 $O(n)$ 。第三，对 $u = B$ 和 $v = YD$ 做 gadget 分解，若分解长度为 L ，则系数级复杂度约为 $O(Ln)$ ；若考虑 r 个 RNS prime，则约为 $O(Lnr)$ 。第四，执行从 (s_0, s_1) 到 s' 的 key switching。每个 digit 需要处理 $u_\ell b_{0,\ell}$ 、 $u_\ell a_{0,\ell}$ 、 $v_\ell b_{1,\ell}$ 、 $v_\ell a_{1,\ell}$ 等小环乘法，因此主成本约为 $O(L \cdot \text{Mul}_{R_n})$ 。若小环乘法用 NTT 实现，并省略常数与 RNS 细

节, 则 $\text{Mul}_{R_n} = O(n \log n)$, 主复杂度为 $O(Ln \log n)$; 若显式计入 RNS prime 数, 则可写为 $O(Lrn \log n)$ 。

因此, 压缩总复杂度可概括为 $O(N) + O(Ln) + O(Ln \log n)$, 主导项通常是小环 key switching。相比继续留在大环 R_N 中进行后续 NTT 型运算, 压缩后后续多项式运算从 $O(N \log N)$ 降为 $O(n \log n)$ 。由于 $N = 2n$, 比例约为 $n \log n / (N \log N) = \log n / (2 \log(2n))$, 当 n 较大时接近 $1/2$ 。因此, 压缩是否值得取决于后续计算量: 若压缩后立即结束, 则一次 key switching 成本未必划算; 若压缩后还要继续递归 FFT、bootstrapping 子过程、卷积或大量同态乘法, 则半维度小环可能带来收益。

4.9 密钥材料复杂度

大环到小环压缩需要两组切换密钥, 分别用于 $s_0 \rightarrow s'$ 和 $s_1 \rightarrow s'$ 。对每个 $\ell = 0, \dots, L-1$, 每组 key 包含两个小环多项式, 因此总共约需 $4L$ 个小环多项式作为 evaluation key。若每个小环多项式有 n 个系数, 则存储复杂度为 $O(Ln)$; 若考虑 r 个 RNS prime, 则为 $O(Lnr)$ 。如果后续合并还要求从 $s'(X^2)$ 切回某个目标大环私钥 $S(X)$, 则还需要额外的升环 key switching key, 其存储量应另行计入。

4.10 重复压缩-合并的分布备注

压缩-合并不是原始密文的严格逆变换。若记一次复合操作为 T , 则 $T(\text{ct})$ 与 ct 在明文语义上等价, 但通常不是同一个密文。若二者最终绑定在同一私钥下, 则差分 $T(\text{ct}) - \text{ct}$ 是一个加密零的密文。在标准 RLWE 与 key switching 安全假设下, 这类零密文样本不应泄露明文信息。不过, 由于 $T(\text{ct})$ 不是新鲜重加密, 其噪声分布可能带有压缩、合并和 key switching 的操作痕迹。因此本文不声称该过程满足 circuit privacy; 若需要隐藏操作历史, 应额外考虑重随机化或 noise flooding。

4.11 本节总结

在本文抽象模型下, FFT 式奇偶拆分的主要新增噪声来自一次自同构 key switching; 大环到小环压缩的主要新增噪声来自一次从向量私钥 (s_0, s_1) 到标量私钥 s' 的 gadget key switching。压缩输出噪声可概括为 $e_{\text{out}} = \nu_0 + \eta_{\text{ks}}$, 其中 ν_0 是原始噪声偶部, η_{ks} 是压缩 key switching 噪声。复杂度方面, 公开提取、单项式移位和 gadget 分解不是瓶颈, 主导成本是小环 key switching, 抽象复杂度为 $O(Ln \log n)$, 计入 RNS prime 后为 $O(Lrn \log n)$ 。因此, 压缩操作可以理解为: 用一次小环 key switching 的代价, 换取后续计算在半维度小环中进行的机会。

5 可能的优化方向

6 待完成

1. 补 key switching key 的定义和安全假设。
2. 补参数安全提醒: 小环安全性要重新估计。

3. 补 slot 层面的解释，作为另一个视角。
4. 最后整理的时候写成命题/定理 + 证明的形式。、
- 5.

小项:

1. 补密钥材料复杂度的层级版本.
2. 补 failure cases 的分析
3. 重复压缩-合并的分布偏差问题探索。0 差分

参考文献

- [1] J.-S. Coron and T. Seur , “PaCo: Bootstrapping for CKKS via partial CoeffToSlot,” Cryptology ePrint Archive, Paper 2025/886, 2025. [Online]. Available: <https://eprint.iacr.org/2025/886>
- [2] R. Geelen, “Revisiting the slot-to-coefficient transformation for BGV and BFV,” Cryptology ePrint Archive, Paper 2024/153, 2024. [Online]. Available: <https://eprint.iacr.org/2024/153>
- [3] S. Ma, T. Huang, A. Wang, and X. Wang, “Faster BGV bootstrapping for power-of-two cyclotomics through homomorphic NTT,” Cryptology ePrint Archive, Paper 2024/164, 2024. [Online]. Available: <https://eprint.iacr.org/2024/164>
- [4] C. Peikert, D. Zarchy, and G. Zyskind, “High-precision exact FHE made simple, general, and fast,” Cryptology ePrint Archive, Paper 2025/2321, 2025. [Online]. Available: <https://eprint.iacr.org/2025/2321>

A 平凡矩阵抽取路线

本附录说明一种最直接但并不采用的奇偶系数抽取方法。该方法的想法是：先把多项式 $P(Y) = \sum_{j=0}^{2n-1} p_j Y^j$ 的系数看成一个长度为 $2n$ 的向量 $\mathbf{p} = (p_0, p_1, \dots, p_{2n-1})^\top$ ，然后用矩阵直接抽取偶数下标系数和奇数下标系数。

A.1 整体置换矩阵

定义置换矩阵 $Q \in \{0, 1\}^{2n \times 2n}$ ，使得

$$Q\mathbf{p} = (p_0, p_2, \dots, p_{2n-2}, p_1, p_3, \dots, p_{2n-1})^\top.$$

也就是说， Q 把偶数下标系数移动到前 n 个位置，把奇数下标系数移动到后 n 个位置。矩阵形状为

$$Q = \begin{pmatrix} 1 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 1 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 0 & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 1 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 1 \end{pmatrix} \quad Q_{r,c} = \begin{cases} 1, & 0 \leq r < n, \quad c = 2r, \\ 1, & n \leq r < 2n, \quad c = 2(r - n) + 1, \\ 0, & \text{otherwise.} \end{cases}$$

因此 Q 可以分块写成 $Q = \begin{pmatrix} E \\ O \end{pmatrix}$ ，其中 E 是偶次系数抽取矩阵， O 是奇次系数抽取矩阵。

A.2 偶次系数抽取矩阵

偶次系数抽取矩阵 $E \in \{0, 1\}^{n \times 2n}$ 满足

$$E\mathbf{p} = (p_0, p_2, \dots, p_{2n-2})^\top,$$

其矩阵形状为

$$E = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{pmatrix}, \quad E_{i,j} = \begin{cases} 1, & j = 2i, \\ 0, & \text{otherwise,} \end{cases} \quad i = 0, \dots, n-1, j = 0, \dots, 2n-1.$$

A.3 奇次系数抽取矩阵

奇次系数抽取矩阵 $O \in \{0, 1\}^{n \times 2n}$ 满足

$$O\mathbf{p} = (p_1, p_3, \dots, p_{2n-1})^\top,$$

其矩阵形状为

$$O = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 1 & \cdots \\ 0 & 0 & 0 & 0 & \cdots \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & 1 \end{pmatrix}, \quad O_{i,j} = \begin{cases} 1, & j = 2i + 1, \\ 0, & \text{otherwise,} \end{cases} \quad i = 0, \dots, n-1, j = 0, \dots, 2n-1.$$

因此整体上有

$$Q\mathbf{p} = \begin{pmatrix} E\mathbf{p} \\ O\mathbf{p} \end{pmatrix} = \begin{pmatrix} p_0 & p_2 & \cdots & p_{2n-2} & p_1 & p_3 & \cdots & p_{2n-1} \end{pmatrix}^\top.$$

A.4 为什么本文不采用这条路线

此路线在明文线性代数层面非常直接：用 E 和 O 抽取偶数和奇数下标系数即可。但在 CKKS 中，如果同态执行矩阵抽取，需要先将多项式系数搬到槽中 (CoeffToSlot)，再进行槽向量上的矩阵-密文乘法，若后续还要回到多项式表示，还需 SlotToCoeff。

即流程为

$$\text{Enc}(P) \longrightarrow \text{Enc}^*(\mathbf{p}) \longrightarrow \text{Enc}^*(E\mathbf{p}), \text{Enc}^*(O\mathbf{p}) \longrightarrow \text{Enc}(P_{\text{even}}), \text{Enc}(P_{\text{odd}}),$$

其中第一步本质是 CoeffToSlot，最后一步是 SlotToCoeff。两类操作在 CKKS bootstrapping 中非常昂贵，因此仅为奇偶拆分引入代价通常不可接受。

本文做法不是显式搬到槽再用矩阵抽取，而是直接在多项式环中利用

$$P(Y) = P_{\text{even}}(Y^2) + YP_{\text{odd}}(Y^2), \quad P(-Y) = P_{\text{even}}(Y^2) - YP_{\text{odd}}(Y^2),$$

通过 $P(Y)$ 与 $P(-Y)$ 的半和、半差完成拆分，从而绕开昂贵的矩阵抽取路线。